

Rewritten Draft — Sections I, II, III (A-D)

For plagiarism reduction — original content restructured and rewritten in own voice

How to use this: These are full rewrites, not light edits — sentence order, structure, and phrasing have been deliberately changed from your draft so they don't pattern-match against the same sources Turnitin flagged the first version against. Read through and adjust anything that doesn't sound like your own voice, since a checker also compares your paper against itself over time (self-plagiarism across resubmissions is real). Citations are kept only where a specific factual claim traces back to a cited source — verify each one still matches what you meant to cite.

I. INTRODUCTION

Every year, a meaningful fraction of enrolled engineering students leave their program before completing it, and the institutional and personal cost of that attrition is well documented [1]. The frustrating part is that the signs are usually visible well in advance — attendance drifting downward, grades slipping over consecutive terms, fee payments becoming delayed — but by the time a mentor or advisor notices the pattern, it is often through a manual, end-of-semester review, at a point where the window for meaningful intervention has already narrowed considerably.

This is the core limitation of most existing Early Warning Systems (EWS) in higher education: they are built to *detect* risk, not to *act* on it. A model can correctly flag a student as high-risk and still leave the actual work of deciding what to do next — which resource to point them to, which scholarship they might qualify for, what a realistic academic recovery plan looks like — entirely to an already-stretched mentor. The prediction, in other words, is treated as an endpoint rather than the start of a workflow.

Large Language Models offer an obvious way to fill that gap, since they can turn a risk score into a written, specific, readable recommendation. But a general-purpose LLM with no grounding in a specific institution's actual policies, scholarship criteria, or a specific company's recruitment process will confidently produce recommendations that sound plausible and are wrong — a well-known failure mode often referred to as hallucination. Any system that hands raw LLM output to a student without safeguards is trading one risk (a missed intervention) for another (a confidently incorrect one).

This paper describes a system, referred to here as ASRA, built to close that gap directly. Rather than treating prediction and recommendation as two separate tools bolted together, ASRA is designed as a single closed loop: a risk-prediction engine feeds a reasoning agent, the reasoning agent grounds its output in real institutional and placement data before generating anything, and nothing generated ever reaches a student without a human mentor reviewing and approving it first.

Concretely, this work contributes three things. First, a risk-classification engine tuned deliberately to favor recall over precision, on the reasoning that missing a genuinely at-risk student is a far more costly error than occasionally over-flagging one who turns out to be fine, paired with per-student feature attribution so the "why" behind a score is never a black box. Second, an orchestration layer — built on LangGraph rather than a fixed sequence of API calls — that reasons over a student's specific risk profile and only invokes the tools (scholarship checks, resource retrieval, plan drafting) that are actually relevant to that student's situation, meaning two students with different risk profiles can genuinely take different paths through the system. Third, a mentor-authenticated approval interface that sits between every AI-generated recommendation and the student it concerns, so the system augments a mentor's judgment rather than substituting for it.

II. RELATED WORK

A. Predictive Early Warning Systems

Machine learning classifiers have become the default tool for institutional early-warning work, largely because student data tends to be tabular, sparse, and full of the kind of heterogeneous features — attendance percentages, fee status, term-over-term GPA — that tree-based ensemble methods handle noticeably better than deep learning architectures designed for dense or sequential data. Among these, XGBoost has become a particularly common choice for exactly this reason [2].

One design decision worth calling out specifically is how a model's decision threshold gets calibrated. Carballo-Mendivil et al. built an early-warning classifier that deliberately weighted recall above precision, on the basis that in an educational context, a false negative — a student who was actually going to drop out but never got flagged — causes far more harm than a false positive that simply results in an unnecessary check-in [4]. That same reasoning shaped the calibration choices in the risk engine described in this paper.

A second recurring theme in this literature is explainability. Wang and Luo showed that Shapley Additive Explanations (SHAP) can be layered on top of an otherwise black-box classifier to produce per-student, per-feature attribution — not just *that* a student is at risk, but which specific factors are driving that classification [5]. This matters beyond interpretability for its own sake: a risk score with no attached reasoning gives a downstream system (or a human mentor) nothing to act on, whereas knowing that attendance decline is the dominant factor for one student and fee delay is dominant for another allows genuinely different, targeted responses. Despite this, most systems in this space still stop at the prediction. The risk score is the final output; what happens after a student is flagged is left entirely outside the system's scope.

B. Conversational and Adaptive Learning Agents

A separate line of work has focused on the generation side of the problem rather than the prediction side. Adaptive learning systems personalize academic guidance by conditioning

on a student's own performance history, current skill level, and stated goals, adjusting recommended learning paths as those inputs change rather than issuing the same generic advice to every student [6].

The arrival of capable LLMs has pushed this further, since natural-language generation makes it possible to produce guidance that reads as specific and conversational rather than templated. Systems that combine an LLM layer with predictive analytics can move past simply reporting a risk level and instead generate an actual improvement plan tied to that risk. A related subset of this work focuses specifically on career and placement guidance — matching a student's current skills against what a target employer or role actually requires, and surfacing the specific gap between the two as a preparation roadmap rather than generic career advice [6].

C. Gap Addressed by This Work

Taken together, the two threads above cover two separate halves of the problem: accurate, explainable risk classification on one side, and grounded, personalized natural-language generation on the other. What is largely missing from the literature is a system that treats these as one connected pipeline rather than two independent capabilities — where the *output* of the classifier directly determines *how* the generation agent behaves, where that generation is grounded in real retrieved institutional data rather than an LLM's unconstrained output, and where a human mentor sits in the approval path before anything reaches a student. The architecture in this paper is built specifically to close that loop: a shared data contract connects prediction to generation, an agent's behavior branches based on the specific risk factors it receives rather than following one fixed script, and every generated recommendation passes through mentor approval before it becomes visible to a student.

III. SYSTEM ARCHITECTURE

The system is built as four cooperating services rather than one monolithic application, communicating through a shared, versioned JSON contract instead of a shared codebase or shared database. This was a deliberate choice: it lets each service be built, tested, and scaled independently, at the cost of needing to keep the contract itself stable as the system evolves.

A. Risk Prediction Engine

The Risk Engine is the entry point of the pipeline — a stateless FastAPI service that takes a student's academic and behavioral record and returns a risk classification. Internally it runs a pre-trained XGBoost model over the incoming feature set, producing both a categorical risk band and a set of per-student feature attributions describing which inputs contributed most to that classification. Because the service is stateless, it can be queried independently for a single student or swept across an entire cohort without maintaining any session

context, and its output is structured to slot directly into the shared JSON contract consumed by the orchestration layer downstream.

B. Mentor Agent Orchestrator

Where the Risk Engine answers *how at-risk is this student*, the Mentor Agent answers *what should actually be done about it*. It is the component responsible for turning a raw risk classification into a concrete, personalized recommendation, and it does this by combining the risk profile with retrieval-augmented generation and an LLM reasoning layer rather than applying a fixed template.

Structurally, the Mentor Agent is implemented as a LangGraph graph rather than a linear script, meaning its execution path is determined by conditional edges rather than a hardcoded sequence of function calls. A run begins by pulling the student's current risk profile from the Risk Engine over its REST interface — this includes the risk band itself, a confidence value, fee-delay status, placement-related profile fields, and the dominant risk factors behind the classification. Where relevant, the agent separately retrieves the student's selected target company from the roadmap/student-management service, which allows placement guidance to be tailored to a specific employer's expectations rather than generic career advice.

C. Faculty Dashboard Interface

The Faculty Dashboard is the human checkpoint in the system, built as a Streamlit application backed by a persistent PostgreSQL store. Its role is to give mentors visibility into both the raw risk predictions and the AI-drafted recommendations generated in response to them, and to require explicit mentor approval before either is exposed to a student.

Access control is handled deliberately conservatively: a landing page separates the mentor and student experiences entirely, routing students to their own dedicated application rather than giving them any path into the mentor-facing tools. Mentor accounts authenticate against a PostgreSQL credential store using bcrypt-hashed passwords with persistent, cookie-based sessions, and — importantly — mentor accounts are provisioned only by an administrator rather than through open self-registration. This closes off what would otherwise be a meaningful security gap: without it, nothing would technically stop an unverified user from creating a mentor account and approving fabricated recommendations for real students during a live evaluation.

D. Student Roadmap Agent

The Student Roadmap Agent sits at the opposite end of the pipeline from the Risk Engine — it is the component students interact with directly, rather than something operating purely behind the scenes for mentors. Its job is to turn a student's academic profile, risk insights, and stated career goals into concrete, personalized guidance.

It is built from four cooperating parts. A Streamlit-based profile interface surfaces the student's own CGPA, risk status, and recommended next steps. An intent-classification module sits in front of everything else and routes an incoming student query to whichever

specialized module actually applies — roadmap generation, explanatory Q&A, or career guidance — rather than handling every query with the same logic. The roadmap-generation module itself uses generative AI to lay out a structured, time-boxed academic improvement plan, typically spanning a four-week horizon. The career-guidance module works differently: it cross-references the student's current skill set against the specific requirements of their selected target company, surfacing the actual gap between where a student is and what their target role expects, rather than offering generic career advice untethered to any specific employer.

Notes for further plagiarism reduction

1. **Run this version through Turnitin again before assuming it's clean.** Rewriting reduces but doesn't guarantee zero match — some phrases describing very standard technical concepts (e.g. "bcrypt-hashed passwords," "stateless FastAPI service") will always show minor matches since there's only so many ways to say them; that's normal and checkers generally don't penalize short technical phrases like this heavily.
2. **Check the abstract and Section IV-VI too** — you said you're focused on I, II, and III A-D, but if those sections were written in the same pass as the rest, the same close-paraphrasing issue likely exists there as well.
3. **Table I and the equation in III.E are safe** — numeric results and standard formulas (like the softmax equation) aren't plagiarism risks in the way prose is.
4. **Double check your citations still make sense** after the rewrite — I preserved the four in-text citations from your original ([1], [2], [4], [5], [6]) but moved where some of them anchor; verify each one still points at the claim it's meant to support before submitting.
5. **If your teammates wrote other sections independently**, make sure none of you are unintentionally repeating the same description of a shared component (e.g. the JSON data contract) near-verbatim across sections — that shows up as internal self-similarity too.